01 - Model records before querying

Read, predict and model

Name _____ Class _____ Date _____

Identify keys, relationships and a reason to separate tables.

Textbook: database-foundations

Before starting: Activities 01-04 establish the Year 11 foundation. Before Activity 05, complete whole-result aggregate practice in the SQL pathway; grouped reports belong to the Year 12 route. HAVING and LEFT JOIN are supporting extensions with their own preparation. Activity 06 needs prior SELECT/WHERE practice and a disposable database copy. Follow the prerequisites, rather than treating the activity numbers as one beginner lesson.

Use this synthetic dataset throughout the pack. returned is 1 for returned and 0 for outstanding. days is a stored duration, not a live date calculation. Loan.member_id refers to Member.member_id; one member may have zero or many loans. Each loan belongs to exactly one member.

Member ID	Name
1	Ari
2	Bo
3	Cleo

Loan ID	Member ID	Item	Days	Returned
101	1	Camera	3	1
102	1	Headset	5	0
103	2	Camera	2	1
104	2	Tripod	5	0

member_id uniquely identifies a member. A name is not a reliable key: two people can share it. Separating member details avoids changing the same name on every loan.

Equipment-free route

Use the printed tables as record cards. Draw keys and relationships on paper; apply integrity rules manually. Keep these tables beside later SQL activities.

Explain and transfer

Annotate one step of the worked example in your own words. Change one supplied input, record a new prediction and show how the rule produces it. Label a hand trace separately from a software observation.

01 - Model records before querying

Independent practice and evidence

Name _____ Class _____ Date _____

A. Independent model

Draw both entities, underline primary keys and mark the foreign key. Add minimum and maximum relationship participation at each end. Can Cleo exist without a loan?

B. Integrity and dependencies

Would loan (105, 9, Light, 1, 0) be valid with enforced foreign keys? Explain. A flat register repeats a member name on each loan: identify the update anomaly and the dependency that motivates separating Member.

Exit

Could two Loan rows have the same member_id? Could they have the same loan_id? Explain each answer.

02 - Select columns and specify order

Read, predict and model

Name _____ Class _____ Date _____

Read SELECT, FROM and ORDER BY before adding conditions.

Textbook: sql-basics

SQL returns a table. SELECT chooses output columns, FROM chooses source records, and ORDER BY specifies the result order. Without ORDER BY, do not rely on the displayed order. Run setup.sql once in a fresh SQLite database if using software.

```
SELECT loan_id, item
FROM Loan
ORDER BY loan_id;
```

loan_id	item
101	Camera
102	Headset
103	Camera
104	Tripod

The query returns four rows; item alone is not unique. SELECT DISTINCT item FROM Loan ORDER BY item; returns Camera, Headset, Tripod.

Equipment-free route

Write the SQL and construct the result table by hand from Activity 01. Label it a manual result; a school-approved SQLite tool is optional.

Explain and transfer

Annotate one step of the worked example in your own words. Change one supplied input, record a new prediction and show how the rule produces it. Label a hand trace separately from a software observation.

02 - Select columns and specify order

Independent practice and evidence

Name _____ Class _____ Date _____

A. Write a query

Return name and member_id from Member, ordered by name descending. Predict every row, including column order, using Activity 01 tables.

B. Independent transfer

Return loan_id and days ordered by days descending, then loan_id ascending to break ties. Explain why a secondary sort is useful here. Write a second query listing distinct durations in ascending order.

Exit

Does SELECT change the stored rows? Explain SELECT versus ORDER BY.

03 - Filter rows and test boundaries

Read, predict and model

Name _____ Class _____ Date _____

Apply WHERE with inclusive boundaries and explicit Boolean grouping.

Textbook: sql-basics

WHERE filters individual rows before the result is sorted. BETWEEN includes both endpoints. Text literals use single quotes. Parentheses make combined conditions explicit.

```
SELECT loan_id, item, days
FROM Loan
WHERE returned = 0 AND days BETWEEN 3 AND 5
ORDER BY loan_id;
```

The result is (102, Headset, 5) and (104, Tripod, 5). Test the boundary separately: a returned = 0 row with days = 3 would also qualify. This dataset does not by itself demonstrate that lower boundary.

Equipment-free route

Evaluate the Boolean expression on each printed row. Use separate true/false columns for its parts and construct the final table without database software.

Explain and transfer

Annotate one step of the worked example in your own words. Change one supplied input, record a new prediction and show how the rule produces it. Label a hand trace separately from a software observation.

03 - Filter rows and test boundaries

Independent practice and evidence

Name _____ Class _____ Date _____

A. Independent query

Return loan_id and item for Camera loans OR outstanding loans, ordered by loan_id. Then change the condition to require a Camera AND an outstanding loan. Predict both results.

B. Changed test data

Write a query for outstanding loans lasting 2 through 4 days inclusive. Predict the current result, then design three hypothetical test rows that check 1, 2 and 4 days. State whether each should pass. Do not change the shared database yet.

Days / returned	Expected inclusion	Reason
1 / 0		
2 / 0		
4 / 0		

Exit

Write a condition for item names beginning with C using LIKE. Why would = NULL fail to test a missing value in a nullable column?

04 - Join related records without losing meaning

Read, predict and model

Name _____ Class _____ Date _____

Match keys in an INNER JOIN and predict missing matches.

Textbook: database-solution

A join pairs related rows. The ON clause must match the member identifier, not loan_id. A member with two loans produces two result rows; that is expected one-to-many behaviour.

```
SELECT Member.name, Loan.loan_id, Loan.item
FROM Member INNER JOIN Loan
ON Member.member_id = Loan.member_id
ORDER BY Loan.loan_id;
```

The result is Ari/101/Camera, Ari/102/Headset, Bo/103/Camera, Bo/104/Tripod. Cleo is absent because an INNER JOIN requires a matching loan.

Equipment-free route

Place matching Member and Loan cards beside one another. Construct each joined output row by hand; include a no-match card for Cleo.

Explain and transfer

Annotate one step of the worked example in your own words. Change one supplied input, record a new prediction and show how the rule produces it. Label a hand trace separately from a software observation.

04 - Join related records without losing meaning

Independent practice and evidence

Name _____ Class _____ Date _____

A. Independent report

Write a joined report listing member name, loan_id and item only for outstanding loans. Order by loan_id and predict every row.

B. Diagnose a faulty join

A learner joins ON Member.member_id = Loan.loan_id. Predict the row count here and explain the error. Another learner omits the matching condition entirely and produces every member-loan pair: calculate that row count.

Exit / later extension

Why does counting joined rows fail to count distinct members? After INNER JOIN is secure, investigate LEFT JOIN: which extra member would it preserve, and what would the missing Loan columns contain?

05 - Aggregate first, then filter groups

Read, predict and model

Name _____ Class _____ Date _____

Distinguish row filters, grouping and aggregate results.

Textbook: unit4-sql

Start with one table. GROUP BY partitions rows by a chosen key. COUNT(*) counts rows in each group and SUM(days) adds stored durations. Every selected non-aggregate column here is grouped. These rules avoid SQLite-specific bare-column behaviour.

```
SELECT member_id, COUNT(*) AS loans, SUM(days) AS total_days
FROM Loan
GROUP BY member_id
ORDER BY member_id;
```

Result: (1, 2, 8), (2, 2, 7). Cleo has no Loan rows, so no group appears. AVG(days) over all four loans is 15 / 4 = 3.75; COUNT(*) is 4.

Equipment-free route

Group printed loan cards by the chosen key and calculate aggregates. Complete the whole-result aggregate pathway before this activity. Attempt HAVING only after the core grouped report and the supporting HAVING explanation; LEFT JOIN is not needed for the core exit.

Explain and transfer

Annotate one step of the worked example in your own words. Change one supplied input, record a new prediction and show how the rule produces it. Label a hand trace separately from a software observation.

05 - Aggregate first, then filter groups

Independent practice and evidence

Name _____ Class _____ Date _____

A. Independent grouped report

Group by item, showing item, COUNT(*) and MAX(days), ordered by item. Predict each row. Then add WHERE returned = 0 before GROUP BY and recalculate.

B. Supporting extension: HAVING

After grouped reports are secure, return member_id and SUM(days) only for groups whose sum is at least 8. Use HAVING SUM(days) >= 8. Explain why WHERE SUM(days) >= 8 is unsuitable here.

Exit: core grouped-report check

Explain why Cleo has no row in a report grouped from Loan alone. Does a missing group mean that Member 3 is missing from Member? Identify the source table and justify your answer.

06 - Make controlled changes and prove the result

Read, predict and model

Name _____ Class _____ Date _____

Use a fresh copy, specific conditions and rollback while practising changes.

Textbook: database-solution

Work only on a disposable copy. In each SQLite connection, run PRAGMA foreign_keys = ON; before BEGIN and check PRAGMA foreign_keys; returns 1. This setting is not saved in the database file. A transaction groups changes until COMMIT makes them persistent or ROLLBACK cancels them. Preview the target row before changing it. The example marks loan 102 returned, checks it, then rolls back.

```
BEGIN;
SELECT loan_id, returned FROM Loan WHERE loan_id = 102;
UPDATE Loan SET returned = 1 WHERE loan_id = 102;
SELECT loan_id, returned FROM Loan WHERE loan_id = 102;
ROLLBACK;
```

Before: (102,0). Inside transaction: (102,1). After ROLLBACK a fresh SELECT gives (102,0). An UPDATE without WHERE would change all four rows.

Equipment-free route

Use a second paper copy of the tables. Pencil in changes, record before/after rows, then discard the copy for rollback. This models effects but does not prove a database engine enforced a constraint.

Explain and transfer

Annotate one step of the worked example in your own words. Change one supplied input, record a new prediction and show how the rule produces it. Label a hand trace separately from a software observation.

06 - Make controlled changes and prove the result

Independent practice and evidence

Name _____ Class _____ Date _____

A. Independent transaction

In a fresh copy, insert Member (4, 'Dev'), then Loan (105, 4, 'Light', 1, 0). Preview and delete only loan 105. Verify Member 4 still exists, then ROLLBACK the whole transaction. Predict final row counts. Write every statement.

B. Create and test a constraint

Write CREATE TABLE Tag with tag_id INTEGER PRIMARY KEY and label TEXT NOT NULL. In a separate scratch database, predict the outcome of inserting (1, NULL), then (1, Light) with the text correctly quoted. Explain what the two constraints protect.

Exit

List the evidence you would retain before committing an authorised change. Explain why running the same insert twice can fail.